

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	In progress	9
Bug	Operator-blocked	1
Bug	Queued	8
Bug	Ready for testing	3
Task	In progress	12
Task	Queued	29
Task	Ready for testing	2
TOTAL		64

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): COMPLETE THE CAPTCHA FLOW ONCE Session establishment path is DECIDED: the operator drives the interactive CAPTCHA flow at /api/v1/auth/rutracker/captcha then /login once, and the proxy stores the resulting bb_session. The cookie-paste path and the cookies-file autoload path were both offered and NOT chosen. THE BLOCK NARROWS BUT DOES NOT LIFT: the operator must physically drive the flow — no agent can solve a CAPTCHA (§11.4.52 honest

ATM ID	Type	Status	Severity	Description
				operator_attended boundary). AGENT PREP OWED BEFORE THE HAND-OFF, so the operator's single attempt succeeds rather than discovering a broken endpoint mid-flow: verify both endpoints are reachable and correctly wired end-to-end, verify the session is actually PERSISTED after a successful post (not merely accepted), and verify a stored session is actually USED by a subsequent search. Honest limitation to state up front: a stored bb_session expires, so this path requires repeating — that cost was accepted with the decision. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward
BOB-068	Bug	In progress	High	RD2-00: unattributed, unreviewed Auto-commit mechanism pushing to main
BOB-069	Task	In progress	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill
BOB-074	Task	In progress	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix
BOB-077	Task	Queued	High	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): UNKNOWN — INSTRUMENT THE NEXT OCCURRENCE The operator does not currently know which host produces the +0500 Auto-commit fast-forwards, so the

ATM ID	Type	Status	Severity	Description
				three candidate answers (second Claude session / intentional rsync job / stale job) all remain open. DECIDED ACTION: stop hunting a host this session cannot reach (§11.4.6 — remote state is not knowable without access) and instead INSTRUMENT the mechanism so the next occurrence identifies itself. Forensic capture to add: committer identity, hostname, timezone offset, push timing, and the git remote used, recorded at Auto-commit time into a tracked forensic log. ACCEPTANCE: the next Auto-commit event yields a captured record naming its origin host — at which point this item resolves to one of the three original branches with evidence rather than a guess. This is the §11.4.101 reversible-safe move: it costs little, blocks nothing, and converts a recurring mystery into a self-identifying event. This answer is recorded as consumer DATA per §11.4.35 — it is the operator's stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)

ATM ID	Type	Status	Severity	Description
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)
BOB-090	Task	In progress	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack
BOB-093	Task	In progress	High	RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-094	Task	In progress	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	In progress	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Queued	High	OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): DEFER PAST v1.0.0 Go --profile go parity remains a goal but is NO LONGER a v1.0.0 release blocker. RW-10..13 move to a post-1.0 milestone and stop gating the tag. Nothing advertised is removed, so §11.4.122 stays clean and no §11.4.90 Obsolete closure is warranted. The BOB-141 measured fact stands unchanged: the Go container's Dockerfile runs ONE binary binding only :7187; nothing binds :7186 or :7188 despite compose setting PROXY_PORT/BRIDGE_PORT. CLAUDE.md's port map already describes what the container ACTUALLY does and needs no amendment under this decision. This

ATM ID	Type	Status	Severity	Description
BOB-102	Task	In progress	Medium	answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question. Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — GA-19/RW-09: Is –profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION OPERATOR DECISION (2026-08-26, §11.4.66 interactive clarification): KEEP 0.0.0.0 + MANDATORY AUTH GUARD The tunnel STAYS LAN-reachable — no bind address changes to 127.0.0.1. The operator explicitly chose to preserve access from other devices on the network. THE DECISION CARRIES A BINDING OBLIGATION: a permanent §11.4.135 regression guard MUST land that FAILS THE BUILD if any LAN-reachable route ever stops demanding authentication. The operator accepted 0.0.0.0 ON THE CONDITION that guard exists — without it this decision is unprotected and the item is NOT closeable. Guard requirements: enumerate routes from the authoritative source (FastAPI app, Go handlers, boba-jackett) never a hand-maintained list; resolve real auth coverage not a grep for the string ‘auth’ (§11.4.201 real-condition); deliberately-public routes exempt ONLY via a checked-in list with per-entry justification, enumerated as honest gaps never silent; golden-TRUE + golden-FALSE fixtures per §11.4.107(10). This answer is recorded as consumer DATA per §11.4.35 — it is the operator’s stated choice, not an agent inference, and supersedes any prior agent-chosen default on this question.

ATM ID	Type	Status	Severity	Description
				Options not chosen are named above so a future reader does not re-litigate a settled call (§11.4.112(5) bounded-verdict discipline applied to decisions). — prior item text follows — RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence-check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Ready for testing	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golden-bad fixture for the rate-limit detector
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3)
BOB-135	Bug	Ready for testing	Low	Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied / config)
BOB-137	Bug	In progress	High	Merge service on 7187 wedges while the same process still serves 7186 (GIL starvation by one spinning thread)
BOB-141	Task	Queued	Low	CLAUDE.md claims the Go profile serves 7186/7187/7188 but its container binds only 7187 — doc contradicts the Dockerfile
BOB-143	Bug	Queued	Medium	Orphaned .worktrees/ dirs (46M, unresolvable gitdir) pollute gate scan

ATM ID	Type	Status	Severity	Description
				scope and manufacture false BOB-126-class findings
BOB-150	Task	Queued	Low	pre_build_verification.sh invariant labels read N/44 but only 35 invariants are labelled
BOB-151	Task	Queued	Medium	CM-SCRIPT-DOCS-SYNC is a named gate with no implementation, and 24 of 36 scripts have no companion doc
BOB-152	Bug	Queued	Medium	Constitution sweep walks vendored third-party code: 82% of one gate's 38,291 findings come from submodules/
BOB-154	Bug	Ready for testing	Medium	Host venv and production container run different starlette versions (1.4.1 vs 1.6.0) OPERATOR DECISION (2026-08-26, §11.4.66): REPAIR, THEN RE-VERIFY UNTIL STABLE The warm ./start.sh path keeps running the ownership repair against a live stack, but walks, re-verifies, and repeats until a pass finds nothing new. Options not chosen: refuse the repair while the stack is up and direct the operator to –recreate; stop the stack on the warm path too (correct but ends ‘warm’); accept and document the window. WHY THIS MATTERS MORE THAN A NORMAL ITEM: this is the exact defect the whole 002-user-owned-downloads feature exists to end. The –recreate path was already fixed to order precondition -> down -> repair -> up so the walk sees a tree no container can write. The warm path was not, so a container can create a new non-owned file AFTER the repair has passed that directory — the ownership problem leaking back in through the other door. REQUIRED BY THE CHOSEN OPTION, and it is the hard part: the loop MUST be BOUNDED, and on giving up it MUST report honestly that ownership is UNPROVEN rather than silently exiting 0
BOB-159	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				(§11.4.201(6) — a loop that stops finding new files because it ran out of iterations returns the same quiet zero as a genuinely clean tree). An active download creating files faster than the walk completes is the realistic non-converging case and must be named in the give-up message. Recorded as consumer DATA per §11.4.35 — the operator’s stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. — prior item text follows — Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:01:17Z Reported-By: T041 independent review (IMPORTANT-2), partially remediated What (the report, verbatim): The –recreate path was fixed this round: run_ownership_precondition -> stack_down -> run_ownership_repair -> stack_up, so the repair walks a tree no container can write to. That ordering is now asserted behaviourally by three new checks in tests/unit/test_start_reload_recreate.sh, each killed by its paired 1.1 mutation (11/11 RED-OK). The WARM path is NOT covered by that fix and this item tracks the remainder. On a warm ./start.sh the gate runs before THIS invocation brings anything up, but if the stack is ALREADY running, containers write while the repair walks. A container can then create a new non-operator-owned file BEHIND the walk, after which the completion marker records complete over a tree that is not, and those stragglers are never repaired without a manual –force. BOUNDED, not dismissed: after any successful pass the marker is valid for the scope fingerprint and the repair short-circuits without walking (marker_is_valid), so the window requires a stale-or-absent marker AND a live stack

ATM ID	Type	Status	Severity	Description
				together. Declaring a new scope entry re-arms the marker, which is exactly how that combination arises in practice. NOT DONE THIS ROUND, with the reason stated rather than hidden: the clean guard is a cheap marker-only probe mode on scripts/ownership_repair.sh that answers has-this-scope-been-repaired without walking the tree. The existing –dry-run cannot serve: it deliberately SKIPS the marker fast-path (FORCE=0 && DRY_RUN=0 guard) and always walks, so using it as a warm-start probe would walk the tree twice on every start. Adding that mode requires editing ownership_repair.sh, which another stream was actively editing during this round; duplicating marker_is_valid into start.sh instead would be a near-identical fork (11.4.251). Deferred to this item rather than done unilaterally or silently. The misleading comment at the warm-start call site (which claimed the gate runs before any container writes) has been corrected to state this boundary honestly. Affected scope / file-scope manifest: start.sh (warm-start dispatch, run_ownership_gate call site); scripts/ownership_repair.sh (marker fast-path) Reproduction / context: 1. Ensure the stack is UP. 2. Change config/owned_paths.yaml so the scope fingerprint changes (this re-arms the marker by design, data-model E2). 3. Run a warm ./start.sh (no –recreate). 4. The ownership repair walks and chowns the declared tree while containers are still running and able to write into it. Acceptance criteria: A warm ./start.sh cannot complete an ownership repair while a container that writes to a declared location is running. Either the repair is deferred with an actionable refusal

ATM ID	Type	Status	Severity	Description
BOB-161	Task	Queued	High	naming –recreate, or the stack is quiesced for the walk. Asserted behaviourally in tests/unit/test_start_reload_recreate.sh alongside the existing PRECONDITION_BEFORE_DOWN / REPAIR_AFTER_DOWN / REPAIR_BEFORE_UP checks, each with a paired 1.1 mutation that kills it. Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:31:12Z Reported-By: BOB-092 remediation, residual finding What (the report, verbatim): §11.4.69 names CM-NO-FAIL-OPEN-SKIP as one of three mandatory pre-build gates: it audits sink-side probe helpers and FAILs if any code path converts an empty or unreachable response into a PASS-counting SKIP for a feature class with a sink-side probe. This project does not have it. Why this matters now rather than in the abstract: the BOB-092 remediation just removed TWO fail-opens of exactly this class from tests/e2e/test_live_stack_evidence.py — the nnmclub SKIP-on-404 (already removed at 7baef2b) and a surviving sibling at test_iporrents_is_authenticated_in_search that skipped on ‘not authenticated and status != success’ while asserting ‘treating as transient outage’. That assertion was false for rejected credentials (upstream_http_403) and for a broken container (plugin_env_missing), both of which are definitive product failures the product already classifies via error_type. Two fail-opens of one class, found one at a time, is the §11.4.146 extend-to-all-cases signal and the §11.4.238 coverage-escape signal together: the regime did not surface either, an agent reading code did. The remediation’s own guards are AST-structural and lethal under three

ATM ID	Type	Status	Severity	Description
				discriminating mutations, but they LIVE IN THE FILE THEY GUARD, so deleting that file evades them entirely. A pre-build gate is the durable home because it audits the corpus rather than one file. Honest boundary (§11.4.6): this item does NOT claim the two remediated fail-opens are unguarded — they are guarded, with runtime evidence (13 passed in 97.36s against the live stack). It claims the CLASS has no corpus-wide detector, so the next instance in a different file is invisible again. Affected scope / file-scope manifest: scripts/pre_build/ (gate absent); tests/e2e/test_live_stack_evidence.py (guards currently live inside the file they guard) Reproduction / context: grep -rl 'CM-NO-FAIL-OPEN-SKIP' scripts/ tests/ returns exactly ONE hit, and it is tests/e2e/test_live_stack_evidence.py — the file the guards live in, not a gate. Control needle: the same query for CM-OWNERSHIP-INVARIANTS (a gate that does exist) returns 3 files, so the instrument can see gate tokens and the single hit is a real absence, not a blind zero (§11.4.201(7)(b)). Acceptance criteria: A scripts/pre_build/ gate named CM-NO-FAIL-OPEN-SKIP exists, is wired into scripts/pre_build_verification.sh, and audits sink-side probe helpers for code paths converting an empty/unreachable/error response into a PASS-counting SKIP for a feature class that HAS a sink-side probe. It ships a golden-TRUE fixture (a real fail-open -> gate FIRES) and a golden-FALSE-with-carrier (an honest topology/geo skip, and a comment merely MENTIONING the phrase -> gate MUST NOT fire), per §11.4.107(10)/§11.4.201(1). Paired §1.1 mutation makes the gate FAIL before the gate is trusted.

ATM ID	Type	Status	Severity	Description
BOB-162	Task	Queued	Medium	Reported-Via: §11.4.202 reporting directive task on 2026-08-21T19:35:02Z Reported-By: BOB-106/BOB-107 remediation, residual What (the report, verbatim): The TESTS for both guards are now executed: pre-build invariant 30's glob was extended from tests/unit + tests/pre_build to also cover tests/hooks. MEASURED before/after: the glob expanded to 27 suites with 0 under tests/hooks while 3 existed there; it now expands to 30 with 3 under tests/hooks. All three pass. But the GUARDS themselves are still invoked by nothing. check-brief-inputs.sh is honestly conductor-run rather than automatic: a brief's required-input list lives in free-form prose that the Agent tool's structured input does not expose, so an automatic prompt-scraping gate would itself be an unproven pattern-match. Its seam is the dispatch procedure, and that is a documentation + habit change, not a hook. unattributed-commit-guard.sh is different and is why this item exists. Run against real history it names 14 violating commits since the last tag (and 21 bare Auto-commit subjects exist across all refs). Wiring it into the pre-build or commit seam AS-IS would therefore refuse every commit immediately, on pre-existing debt nobody can fix in the moment. That is precisely the 11.4.224(E) brownfield-adoption shape, and 11.4.66/11.4.122 put the choice with the operator, not with an agent inventing a ratchet. The options, stated so the decision is a decision and not a default: (a) immediate hard floor – the seam refuses until all 14 are attributed; (b) one-time monotone-decrease ratchet (the 11.4.135 pattern) – snapshot 14 as the baseline, the count may fall and may never rise, so day one is green and the disease cannot spread; (c) changed-

ATM ID	Type	Status	Severity	Description
				commits-only – gate only commits created from now on, with a scheduled deadline for the historical 14; (d) report-only for a stated period, then escalate. Recommendation, with the reasoning rather than just the pick: (b). It is the pattern this constitution already uses for exactly this situation, it makes the debt visible and bounded immediately, and it cannot be satisfied by deleting the guard's name (11.4.227 closes that gaming channel). But it is the operator's call and this item is not closed until that answer is recorded. Honest boundary (11.4.6): this item does NOT claim the 14 commits are harmful in themselves – it claims their ATtribution is absent and that nothing currently prevents the 15th. Affected scope / file-scope manifest: scripts/hooks/unattributed-commit-guard.sh; scripts/hooks/check-brief-inputs.sh; scripts/pre_build_verification.sh; scripts/commit-push-all.sh Reproduction / context: Both guards run correctly by hand and are covered by passing tests (9/9 and 15/15, each proven non-bluff against a no-op stub). Neither is invoked by scripts/pre_build_verification.sh or scripts/commit-push-all.sh, so neither exerts standing detection pressure (11.4.226: a guard with no execution seam is not coverage; registration is not coverage). Acceptance criteria: Each guard is invoked by a named seam, OR carries a registered deferral pointing at this item. For unattributed-commit-guard.sh specifically, the operator has answered the adoption question below and the answer is recorded as consumer DATA – never an invented ratchet. Reported-Via: §11.4.202 reporting directive bug on 2026-08-21T19:56:53Z
BOB-164	Bug	In progress	Medium	

ATM ID	Type	Status	Severity	Description
				Reported-By: BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run What (the report, verbatim): This is a REAL user-facing defect, not a test-tuning artifact, and it was found by the automated regime rather than by a human squinting at the page – which is exactly the 11.4.238 posture the project is aiming for. The static-grep oracle would NOT have found it. Measured: the served root ships an empty pre-hydration, so any check reading the raw HTML audits a page nobody sees. The violation only exists in the hydrated DOM, which is why 11.4.170 requires a rendered oracle and forbids value-equality assertions as the proof a UI is correct. Severity reasoning, stated rather than assumed: this is user-visible and affects the primary dashboard heading, but it degrades legibility rather than breaking function, and the surface is operator-facing rather than public. Medium, not High. The failing test was left FAILING on purpose (11.4.238) instead of silenced or marked xfail. tests/ux/ currently reports 1 failed, 16 passed; that 1 is this defect. Anyone reading a red UX suite should read it as this item, not as flakiness – and when this is fixed the suite goes fully green, which is the signal that it is closed. HONEST SCOPE LIMIT (11.4.6): only the dashboard landing view was scanned. The /jackett/* sub-routes and the ng-serve-hosted :4200 route set were NOT audited – the commands to close both are recorded in docs/testing/ux_accessibility.md. So this item's 21 nodes are a floor, not a total. Affected scope / file-scope manifest: the Angular dashboard served at http://localhost:7187/ (same compiled SPA as frontend/); production component CSS, not test files

ATM ID	Type	Status	Severity	Description
BOB-168	Task	In progress	Low	Reproduction / context: Run tests/ux/test_live_dashboard_accessibility.py against the running merge service. axe-core v4.13.0, scanning a real Playwright-rendered JS-hydrated DOM, reports color-contrast violations on 21 nodes. Measured pairs: .brand / h1 text #9d001e on background #3c3f41 = 1.43-1.62:1 (WCAG AA large-text floor is 3:1); tagline #808080 on #3c3f41 = 2.68:1 (body-text floor is 4.5:1). Acceptance criteria: axe-core reports ZERO color-contrast violations against the live rendered dashboard, with the fix made in production component CSS rather than by relaxing the assertion or excluding the rule (11.4.120: reconcile to the correct mechanism, never weaken the check). The existing tests/ux/ suite is the guard and already fails today, so the RED is captured – closure requires it flipping GREEN against the live surface, which is runtime-class evidence per 11.4.226. WHAT. scripts/run_all_challenges.sh:66 lists “scaling_horizontal_challenge.sh” in its challenge set. challenges/scripts/scaling_horizontal_challenge.sh does not exist: \$ sed -n ‘66p’ scripts/run_all_challenges.sh “scaling_horizontal_challenge.sh” \$ ls challenges/scripts/scaling_horizontal_challenge.sh ls: cannot access ...: No such file or directory VERIFIED independently, not taken on report — surfaced by the BOB-109 scaling agent and re-checked here by direct invocation. WHY IT MATTERS. Whether this is cosmetic or a §11.4.201 gate-honesty defect depends entirely on how the runner treats a missing entry, and that is the first thing to determine: if it SKIPS silently, the challenge bank advertises

ATM ID	Type	Status	Severity	Description
				coverage it does not have (a §11.4.266 claim-vs-reality row with no passing challenge behind it); if it FAILs, the runner is permanently red for a reason unrelated to the system under test, which trains readers to ignore it. Neither outcome is acceptable; they need different fixes. ACCEPTANCE. (a) Determine and record the runner's actual behaviour on the missing entry by invoking it, not by reading it. (b) EITHER author the challenge, OR remove the entry — with §11.4.124 discipline: check git history for whether it once existed and was deleted, since a silently-dropped challenge is the more interesting defect. (c) If the runner silently skips missing entries, that is its own finding: a missing challenge must be loud (§11.4.3 SKIP-with-reason at minimum), never absent-and-quiet. SEVERITY. Low as a defect, but it sits on the challenge-coverage seam, so (c) may deserve its own item. CORRECTION 2026-08-22 (§11.4.6) — THIS ITEM'S PREMISE WAS FALSE, and the error was mine. Recorded openly rather than quietly edited. scaling_horizontal_challenge.sh EXISTS. It is at submodules/challenges/challenges/scripts/scaling_horizontal_challenge.sh, executable, 3107 bytes, dated Aug 7. So do all 14 other entries and the meta-runner. It was never deleted: added to the roster at 1c959ba, the challenge itself added in the submodule at 873c0b1, and a pickaxe search across both repositories and all branches finds ZERO deletions. HOW I GOT IT WRONG. run_all_challenges.sh reads submodules/challenges/challenges/scripts/. I checked challenges/scripts/ — a DIFFERENT directory belonging to a DIFFERENT, glob-based runner. Two similarly-named directories; I measured

ATM ID	Type	Status	Severity	Description
				the wrong one. That is a §11.4.201(9) field-identity error. Worse, I wrote that it was “verified by invocation”: I had listed a directory and never run the aggregator, so the claim overstated the evidence class as well as getting the answer wrong. THE LESSON, which generalises past this item. In the SAME commit I correctly caught a sibling agent making this exact class of error on BOB-167 — a zero-hit produced by searching one symbol name — and applied a control needle there. Then I omitted the needle one paragraph later on my own measurement. The distinction that would have caught it: A CONTROL NEEDLE PROVES THE INSTRUMENT CAN SEE; IT DOES NOT PROVE IT IS POINTED AT THE THING UNDER TEST. §11.4.201(7) (b) as commonly applied guards blindness; it does not guard mis-aiming. The needle for THIS query would have been to resolve the path the runner itself reads, not to confirm that some directory listing works. WHAT IS ACTUALLY WRONG, and it is worse than what I filed. The missing-entry path runs no challenges by definition, so it can be exercised safely. Run against a checkout where the submodule is absent — the ordinary state of a clone made without –recursive — the REAL runner (byte-identity sha-asserted, not a replica) reports: PASS: 0 FAIL: 0 SKIP: 16 TOTAL: 16 OBSERVED EXIT CODE: 0 An ENTIRELY UN-RUN BANK REPORTS SUCCESS. Not one dangling entry tolerated — the whole suite silently passes without executing anything. That is §11.4.201(6) false-null verbatim, and it is observed-reachable rather than constructed (§11.4.115(G)). REVISED ACCEPTANCE. (a) ANSWERED, though not as filed: the runner reports a loud, counted SKIP and then never blocks, because the exit

ATM ID	Type	Status	Severity	Description
BOB-169	Bug	In progress	Medium	expression reads only the FAIL count. (b) The dangling-entry half is VOID — nothing is dangling. (c) The real work is the exit contract: a roster entry the runner cannot execute must not be indistinguishable from a healthy run. See the decision recorded in docs/qa/BOB-168/decision_missing_vs_skip.md. RELATED, REPORTED NOT FIXED: scripts/pre_build_verification.sh:1792 carries the same SKIP/MISSING conflation one level up, for the other runner. Filed separately. WHAT. The §11.4.65 markdown-export mandate requires every in-scope doc to ship .html/.pdf twins. 286 of the 326 .html files under docs/ (excluding dist/ and node_modules/) are pandoc FRAGMENTS — they begin at WHAT: the CM-PLUGIN-COUNT pre-build gate takes 189 SECONDS to verify 8 documented counts. Measured 2026-08-25: exceeded a 90s probe budget (rc=124), then a bounded re-run completed at 'rc=0 wall=189s' - it PASSES, it is simply slow. CAUSE, MEASURED NOT GUESSED: the marker loop at check_cm_plugin_count.sh:273-290 iterates EVERY LINE of every governed document and spawns a pipeline per line. Per iteration: 'printf grep -oE wc -l tr -d' (4 processes) plus 'printf sed -n' (2) = ~6. Governed docs are AGENTS.md 689 + CLAUDE.md 494 + docs/features/Status.md 670 = 1853 lines, so ~11,118 process spawns, measuring ~102 ms/line. Classic per-line-subprocess shell trap. THE IRONY WORTH PRESERVING, because it explains why nobody simplified it: the in-code comment shows 'oE wc -l' was chosen DELIBERATELY over 'grep -c' after measuring the §11.4.201(12) footgun - on this host (ugrep 7.8.4) 'grep -coE' returned
BOB-196	Task	Queued	—	

ATM ID	Type	Status	Severity	Description
BOB-197	Bug	In progress	—	3 at top level but 1 inside a ‘set -euo pipefail’ subshell, a context-dependent reading. The correctness fix is right and must be kept; it is the PER-LINE application of it that costs the three minutes. HYPOTHESIS REFUTED, recorded so nobody re-walks it: this is NOT worktree traversal. The gate’s find is scoped to \$PLUGINS_DIR (:224), so the five agent worktrees are outside its walk. That guess was raised on BOB-194 and is dead. LIKELY FIX (unverified, stated as a candidate not a conclusion): pre-filter with ONE grep for lines containing ‘CM-PLUGIN-COUNT:’ before entering the loop, reducing ~1853 iterations to the ~8 lines that carry a marker, while leaving the per-line parsing logic - and its footgun-avoiding form - completely unchanged. WHY IT MATTERS beyond impatience: this gate is on the pre-build critical path, and CLAUDE.md leans on it as the mechanical authority for the three distinct plugin rosters (43 curated / 43 engines / 12 bootstrap) whose conflation was BOB-149. A gate slow enough to tempt anyone into skipping it protects nothing. ACCEPTANCE: runtime reduced with the counts and the footgun-avoiding parse preserved byte-for-byte in behaviour, proven by the existing paired mutation still biting plus a before/after wall-clock recorded. §11.4.6: nothing here questions the gate’s VERDICTS - only its runtime. OPERATOR DECISION (2026-08-26, §11.4.66): GENERATE THE TOKEN AND ARM IT The conductor generates a 32-byte random token and writes BOBA_API_TOKEN into .env; the operator configures clients with it. DONE 2026-08-26: token generated via secrets.token_hex(32), appended to .env

ATM ID	Type	Status	Severity	Description
				with a comment naming BOB-197 and the routes.py:83 env-gating fact, chmod 600, value shown to the operator ONCE and never written to any other file. Safety verified BEFORE writing (§11.4.30/§11.4.10): .env matched .gitignore:27 '*env' and was untracked (needle-proven — the same matcher confirmed docker-compose.yml IS tracked, so the negative is real); a §9.2 pre-op backup was taken and is itself ignored; post-write re-check confirmed .env still ignored and absent from git status. Pre-store audit found the only tracked BOBA_API_TOKEN= occurrences are the docker-compose passthrough \${BOBA_API_TOKEN:-} and prose in the leak-audit challenge — no literal value has ever been committed. STILL OWED, and the item stays open until it lands: the BOOT-TIME INVARIANT (§11.4.254) that refuses to start LAN-bound when the token is unset or empty. Without it the arming is a state fix, not a defect fix — §11.4.226(5) is explicit that a state-only repair cannot close a defect item. Options not chosen: operator sets it themselves with the invariant blocking until then; invariant warns first and blocks on a named date. Recorded as consumer DATA per §11.4.35 — the operator's stated choice, not an agent inference. Options not chosen are named so a future reader does not re-litigate a settled call. --- prior item text follows --- OPERATOR DECISION (2026-08-26): arm BOBA_API_TOKEN and keep 0.0.0.0, backed by a BOOT-TIME invariant that refuses to start LAN-bound with the token unset. This item IS that invariant. MEASURED FACT, not inference. download-proxy/src/api/routes.py:83 require_api_token() reads BOBA_API_TOKEN at request time and

ATM ID	Type	Status	Severity	Description
				returns immediately when unset or empty. Its own docstring states it plainly: "BOBA_API_TOKEN unset/empty -> return (OPEN). This is the DEFAULT and preserves the current no-auth contract + dev workflow". docker-compose.yml:237 passes BOBA_API_TOKEN=\${BOBA_API_TOKEN:-}, so the container inherits empty unless the operator declares it. The operator .env does NOT declare it, needle-verified per 11.4.201(7)(b): the same matcher hit QBITTORRENT_DATA_DIR in the same file, so the miss is a real absence and not a blind read. The qbittorrent-proxy container was RUNNING and LAN-bound at measurement time. Consequence: the ten routes the BOB-102 guard resolves as auth-wired are open in practice on a default deploy. WHY A STATIC GATE CANNOT COVER THIS. The BOB-102 gate asserts static route WIRING - that a Depends(require_api_token) marker exists on handler, decorator or router. It cannot assert runtime ARMING, which depends on an env var read per request. Claiming the static gate covers it would be exactly the bluff the gate exists to prevent (11.4.262: machine evidence must match the layer it claims). This is a boot-time invariant class per 11.4.254. ACCEPTANCE: a boot-time check that, when the listener is LAN-bound (not loopback), refuses to start with a non-zero exit and a named cause if BOBA_API_TOKEN is unset or empty, so the open state becomes UNREACHABLE rather than the default. Paired 1.1 mutation: remove the check, prove the service starts LAN-bound-and-open. Golden-FALSE per 11.4.201(1): a loopback-bound listener with the token unset must NOT be refused, or the check becomes a false-positive refusal blocking

ATM ID	Type	Status	Severity	Description
BOB-211	Bug	Queued	minor	legitimate dev work. 11.4.238 COVERAGE-ESCAPE NOTE: found by a subagent reading source during BOB-102 guard construction, NOT by the automated QA regime - which is itself the defect class 11.4.238 names. The boot-time check IS the new automated check that would have caught it. STATUS: LATENT on this host (measured: all six declared locations sit on ONE btrfs mount, fully ownership-expressive) and OPERATOR-GATED for verification. Retained from the dismissed BOB-206 rather than discarded with it, because the scenario is well-founded elsewhere: the declared mount is a udisks auto-mounted REMOVABLE NVMe, so another operator's portable drive being exFAT or NTFS is entirely ordinary. TWO RESIDUALS: (1) REPAIR-LAYER DIAGNOSIS. On a uid-flattening mount whose flattened uid is NOT the operator, probe_location correctly REFUSES (verified: uid=0 and uid=100999 both refuse). But it then hands ownership_repair.sh a chown that will return EPERM, and nothing in the repair path is fstype-aware — so the operator sees a permission failure whose real remedy is a REMOUNT OPTION, which no message says. Reasoned, NOT proven: the shim used to verify BOB-206 models the stat READ, not the chown WRITE, and cannot settle whether chown actually fails. (2) MODE CONTRACT. vfat flattens MODE via fmask/dmask, so the preserve_mode: true / 600 contract on .env and config/boba.db would be silently unenforceable there. §11.4.10-adjacent: a credential file believed to be 600 that is world-readable by mount option is a leak the mode check cannot see. WHY OPERATOR-GATED: a genuine end-to-end

ATM ID	Type	Status	Severity	Description
BOB-219	Bug	Queued	major	RED needs a real uid-flattening mount. The privilege-free route is bindfs -u (FUSE; fusermount IS present and user_allow_other IS set) — but bindfs is ABSENT on this host, as are mkfs.vfat and losetup. So verification needs either a bindfs install or a privileged loopback mount. Neither was attempted (§11.4.21 — high-blast-radius host mutation is operator-gated). ALSO RECORDED: NO test anywhere exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/loopback/mkfs/losetup/bindfs/fusermount all measure 0 across the three ownership test files, needle-proven (probe_location = 3 hits through the same instrument). ACCEPTANCE: (1) operator decides whether to install bindfs or provide a privileged loopback so the RED can be built; (2) if verified, the repair path gains fstype-aware diagnosis naming the remount remedy; (3) the mode contract either detects mode-flattening mounts and refuses, or documents honestly that it cannot be enforced there. DISCOVERY CHANNEL (§11.4.238): retained residual from the BOB-206 verification stream. WHAT, conductor-verified: docs/guides/tracker-credentials.md is TRACKED. Its §11.4.65-mandated export twins docs/guides/tracker-credentials.html and .pdf both EXIST ON DISK, are NOT tracked, and ARE ignored — matched by the .gitignore <i>credentials</i> deny-all at :31. The .md is rescued by an explicit allowlist entry at :38; nobody added entries for the twins. WHY IT IS A LIVE VIOLATION AND NOT A HYPOTHETICAL: §11.4.65 requires every in-scope Markdown document to carry synchronized .html/.pdf siblings, and §11.4.212 requires every §11.4.65-scope doc to be reachable from the README.

ATM ID	Type	Status	Severity	Description
				Two artifacts that exist locally but can never be committed are, from any fresh clone, ABSENT — so every clone of this repository has a tracker-credentials guide with no exports, while the authoring host looks complete. The divergence is invisible on the machine that would notice it. WHY NOBODY SAW IT: this is a §11.4.201(6) FALSE-NULL of the BOB-212 class. git status never listed the twins, so no gate and no author had a signal. The blind instrument and the clean tree return the identical quiet zero. ANOTHER INSTANCE OF THE SAME MECHANISM, filed here rather than separately because it is one fix: docs/qa/BOB-124/evidence/loginctl_user_state.txt is swallowed by *_user* at .gitignore:~55 and is therefore uncommitted §11.4.83 QA evidence. Its SIBLINGS IN THE SAME DIRECTORY — user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log — ARE tracked, saved only by the robust glob negation ! docs/qa/**/*log. That one file is lost purely because it is .txt rather than .log. The pattern is exact: robust glob negations hold; per-file and per-extension rescues leak. ACCEPTANCE: (1) both twins become trackable and are committed alongside their .md; (2) the BOB-124 .txt evidence likewise; (3) whichever BOB-212 fix direction is chosen, it MUST cover these — a fix that closes the future-swallow while leaving these three artifacts permanently uncommittable has addressed the mechanism and not the damage; (4) a check that a tracked .md in §11.4.65 scope has COMMITTABLE twins, so this class cannot recur silently. DISCOVERY CHANNEL (§11.4.238): found by the BOB-212 blast-radius sweep — and only on its SECOND pass. The first pass filtered

ATM ID	Type	Status	Severity	Description
BOB-223	Bug	Queued	major	by a source-extension set that omitted .pdf, which hid this finding entirely; the agent re-ran with no extension filter over all 420 non-artifact paths and recorded the blind spot rather than shipping the first number. Worth keeping: an extension allowlist is itself a false-null generator, which is the same shape as the defect being investigated. WHAT, both conductor-verified with control needles: CM-SCRIPT-DOCS-SYNC occurrences in scripts/ pre_build_verification.sh = 0 (control needle CM-MARKDOWN-EXPORT-SYNC = 6, so the instrument sees). §11.4.44 revision-header enforcement likewise 0 hits (control needle 3). Neither §11.4.18's companion-doc mandate nor §11.4.44's revision header is enforced by any invariant. THE PERVERSE CONSEQUENCE, MEASURED: a new script with NO companion doc passes the gate. Writing the companion doc that §11.4.18 MANDATES creates docs/scripts/.md, which invariant 16 (CM-MARKDOWN-EXPORT-SYNC) then requires to have .html and .pdf twins — and their absence is a BLOCKING failure. This session hit it exactly: docs/scripts/ test_gitignore_swallow_is_loud.{html,pdf} are missing and are two of invariant 16's six violations. So the gate's incentive gradient points AWAY from the constitution: complying with §11.4.18 is punished, ignoring it is free. That is worse than an ungated rule — it is a rule the machinery actively discourages. §11.4.227 IS THE GOVERNING ANCHOR: a named gate that exists only in prose is gate debt, and the ledger of unimplemented CM-* names must be monotone-decreasing. CM-SCRIPT-DOCS-SYNC is named in §11.4.18's

ATM ID	Type	Status	Severity	Description
				own text and implemented nowhere — one concrete row of exactly the 58%-unimplemented population §11.4.227 was minted to shrink. ACCEPTANCE: (1) either CM-SCRIPT-DOCS-SYNC is implemented, or it is registered as a deferral pointing at a tracked §11.4.197 item — silent absence is what §11.4.227 forbids; (2) same for §11.4.44's header check; (3) whichever is chosen, the incentive inversion is closed: it must never be cheaper to skip a mandated doc than to write one — if invariant 16 will demand twins, the doc-creation path must produce them, or invariant 16 must scope out docs/scripts until it can; (4) a paired §1.1 mutation per gate implemented. HONEST NOTE ON SCOPE: this item does not argue §11.4.18 should be enforced immediately — that is an operator call about gate debt priority (§11.4.66). It argues the CURRENT state is incoherent: an unenforced mandate whose observance triggers a different gate's failure. Either enforce both ends or neither. DISCOVERY CHANNEL (§11.4.238): found by the T042 readiness preflight while explaining why a new file broke invariant 16. Not by the automated QA regime. WHAT: ownership_repair walks a declared root and repairs items whose uid is not the operator. The unit suite seeds a foreign uid only onto FILES and SYMLINKS; interior DIRECTORIES stay operator-owned (case 22 seeds only a foreign declared ROOT, on the failure path). Production first-start repairs exactly the untested shape. MANIFEST: tests/unit/test_ownership_repair.sh seed_tree/seed_wrong; scripts/ownership_repair.sh walk at :994. REPRO: seed a tree whose interior directories
BOB-226	Task	Queued	major	

ATM ID	Type	Status	Severity	Description
				carry uid 100000 via podman unshare, run the repair, observe no automated assertion covers the outcome. WHY UNTESTED: the unprivileged harness cannot create symlinks inside a directory it no longer owns, which cases 8/9/18 require. The DECLARED GAPS note cross-references tests/ownership/test_container_writes_owned_files.py, but that covers the CREATION side (FR-002), not the repair-side walk. ACCEPTANCE: an integration-layer test (where the unprivileged-harness constraint does not bind) that seeds foreign-owned interior directories, runs the repair, and asserts post-state ownership plus mode preservation. Surfaced by the BOB-207 independent review 2026-08-27. WHAT: §11.4.212 makes the main README the canonical entry point for ALL project documentation, with no §11.4.65-scope doc reachable by no link path. docs/scripts/check_cm_lan_routes_authenticated.md is not reachable from README.md. MEASURED 2026-08-27: grep -c for the guide name in README.md returns 0, control-needed against a doc README does link (CONTINUATION returns 1), so the instrument is not blind. ACCEPTANCE: README links the guide directly or transitively, and the doc-link generator covers docs/scripts/ so the next such guide cannot land orphaned. Surfaced by the BOB-102 round-13 remediation and independently verified 2026-08-27. BOB-227 measured 3 of 4 artifacts of the CM-LAN-ROUTES-AUTHENTICATED gate untracked in git; criterion 1 (commit them) was already resolved by an unrelated prior commit before this session, closed 2026-09-23. Criterion 2 was
BOB-228	Task	Queued	minor	
BOB-231	Task	Queued	Major	

ATM ID	Type	Status	Severity	Description
				NOT addressed: ‘a gate asserting that every executable a pre-build invariant invokes is itself tracked’ – a general mechanism preventing this whole CLASS of defect (a pre-build gate’s own implementation files shipping untracked, invisible to a fresh clone, no committed baseline for round-over-round diffs) from recurring for ANY future gate, not merely this one. ACCEPTANCE: (1) enumerate every file path scripts/ pre_build_verification.sh invokes (via bash/timeout/python3 calls to scripts under scripts/pre_build/, plus every tests/ pre_build/.sh and tests/hooks/.sh it runs) – likely via a static grep/parse of pre_build_verification.sh itself, or a runtime trace; (2) assert every one of those paths is git-tracked (git ls-files –error-unmatch); (3) wire this as a new pre-build invariant so a future untracked gate implementation is caught immediately, not discovered independently weeks later; (4) a RED test creating an untracked fake gate-invocation target and asserting the new check fails on it, GREEN once the mechanism exists and the fake target is either removed or tracked.